Human Development Index (HDI) Predictor Engine

Project Description

The Human Development Index (HDI) Predictor Engine leverages machine learning to forecast a country's overall development level. The platform enables researchers, analysts, and policymakers to study how improvements in specific socio-economic dimensions impact a nation's overall progress.

The application includes an **HDI Prediction Module**, which takes core indicators — Life Expectancy at Birth, Mean Years of Schooling, and Gross National Income (GNI) Per Capita — and predicts a country's overall HDI score (ranging from 0.0 to 1.0).

It also features a **Dynamic Custom Model Training Module**, allowing users to upload updated CSV datasets directly from the browser to retrain the Linear Regression engine in real time. Deployed publicly via Vercel as a serverless application, the platform provides a simple, responsive, and glassmorphism-styled interface that eliminates local environment dependencies.

Live Application: https://hdi-predictor-sigma.vercel.app/predict

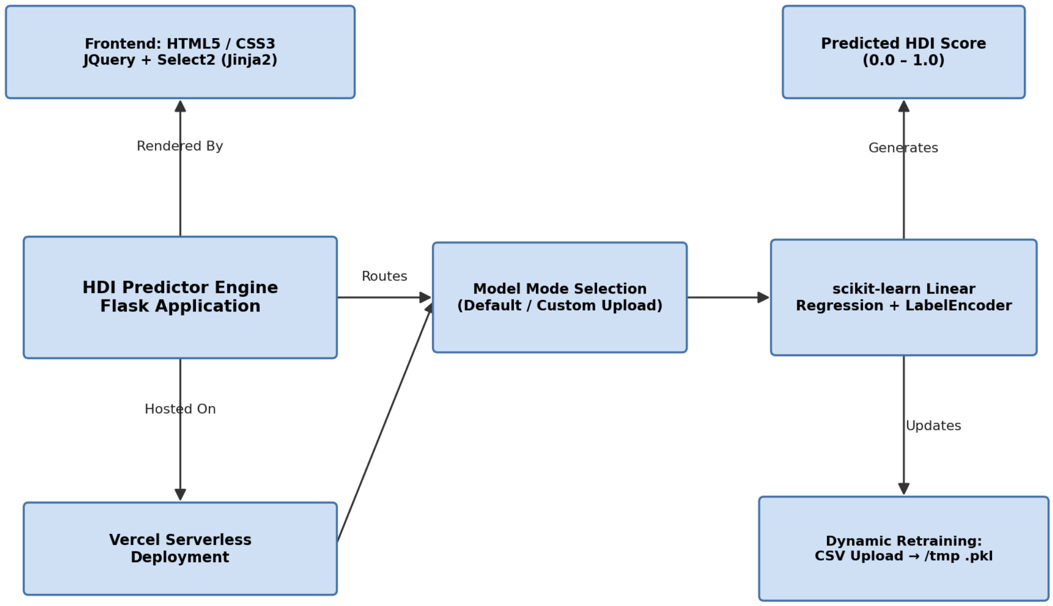
User Scenarios

Scenario 1 (Standard Prediction): An international development analyst selects a country (e.g., India) from the searchable dropdown, inputs a hypothetical increase in education years (e.g., changing schooling from 12 to 16 years), and immediately views the predicted HDI score to evaluate policy impacts.

Scenario 2 (Dynamic Model Retraining): A researcher obtains the latest annual UN statistical annex. On the homepage, they select "Option 2: Custom Model", upload the raw CSV file, and click "Upload & Train". The system cleans the dataset, fits a new Linear Regression model, and updates the application dynamically.

Scenario 3 (Statistical Exploration): A data analyst opens the Jupyter Notebook (HumDevIndex.ipynb) locally, checks the correlation heatmap of indicators, splits the data, runs evaluations, and studies the training coefficients.

Technical Architecture



- **Frontend:** Built with HTML5 and CSS3 (with responsive glassmorphic cards) rendered via Flask Jinja2 templates, utilizing JQuery and Select2 CDN for the alphabetical searchable country dropdown.
- **Backend:** Flask web framework serving API endpoints, configured with custom /tmp pathing (via tempfile.gettempdir()) to dynamically read/write updated model pickle files on Vercel's read-only serverless filesystem.
- **Machine Learning Engine:** Built on scikit-learn using a multi-variable Linear Regression model and a LabelEncoder for country index mapping, serialized via pickle.
- **Deployment:** Hosted as a serverless Python function on Vercel (https://hdi-predictor-sigma.vercel.app/predict).

Pre-requisites

- **Flask Framework Knowledge:** Flask documentation (https://flask.palletsprojects.com)
- **scikit-learn / Machine Learning Familiarity:** scikit-learn documentation (https://scikit-learn.org)
- **HTML & CSS Skills:** Form inputs, grids, and JQuery integration.
- **Python Programming Proficiency:** Pandas and NumPy for cleaning datasets.
- **Version Control with Git:** Code management.
- **Vercel CLI for Public Deployment:** Vercel documentation (https://vercel.com/docs)

Project Workflow & Milestones

Milestone 1: Model Selection and Architecture

Activity 1.1: Prepare the Dataset

- Clean raw UN tables (HDR25_Statistical_Annex_HDI_Table1.csv), filtering out headers and region summaries, leaving 190+ clean rows saved as Dataset/HDI.csv.

Activity 1.2: Select the Model

- Use a Jupyter Notebook (Training/HumDevIndex.ipynb) to select variables, fill missing values, and serialize model files (api/HDI.pkl, api/Encoder.pkl).

Activity 1.3: Define the Architecture

- Set up the serverless directory structure mapping all traffic to api/index.py using vercel.json.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Features

- **Option Selection Cards:** A clickable dual-card selector for Default vs Custom model modes.
- **Centralized Action Button:** A main proceed button at the bottom that is disabled until a model option is selected.
- **Select2 country search bar:** Dynamic country search options alphabetized automatically.

Activity 2.2: Implement Flask Backend

- Map routes for /, /prediction_page, /train_custom, and /predict.

Milestone 3: api/index.py Development

Activity 3.1: Writing Main Application Logic

- Load models dynamically. Check if a newly trained model exists in Vercel's writable /tmp folder; if not, fall back to default pre-trained files inside the api/ directory:

```
with open("api/HDI.pkl", "rb") as f:

model = pickle.load(f)
```

- Parse input arguments, convert them to floats, shape them into a NumPy feature array, run inference, and return the predicted value.

Milestone 4: Frontend Development

Activity 4.1: Designing the User Interface

- Design a card selection grid on home.html and clean inputs on indexnew.html with client-side limits (e.g., Life Expectancy: 0 to 120, Schooling: 0 to 30 years).

Activity 4.2: Handling Form Submission

- Standard form POST submission routes inputs back to /predict for server-side HTML rendering.

Milestone 5: Deployment

Activity 5.1: Local Testing

- Activate environment, install dependencies (pip install -r requirements.txt), and run local server (python api/index.py).

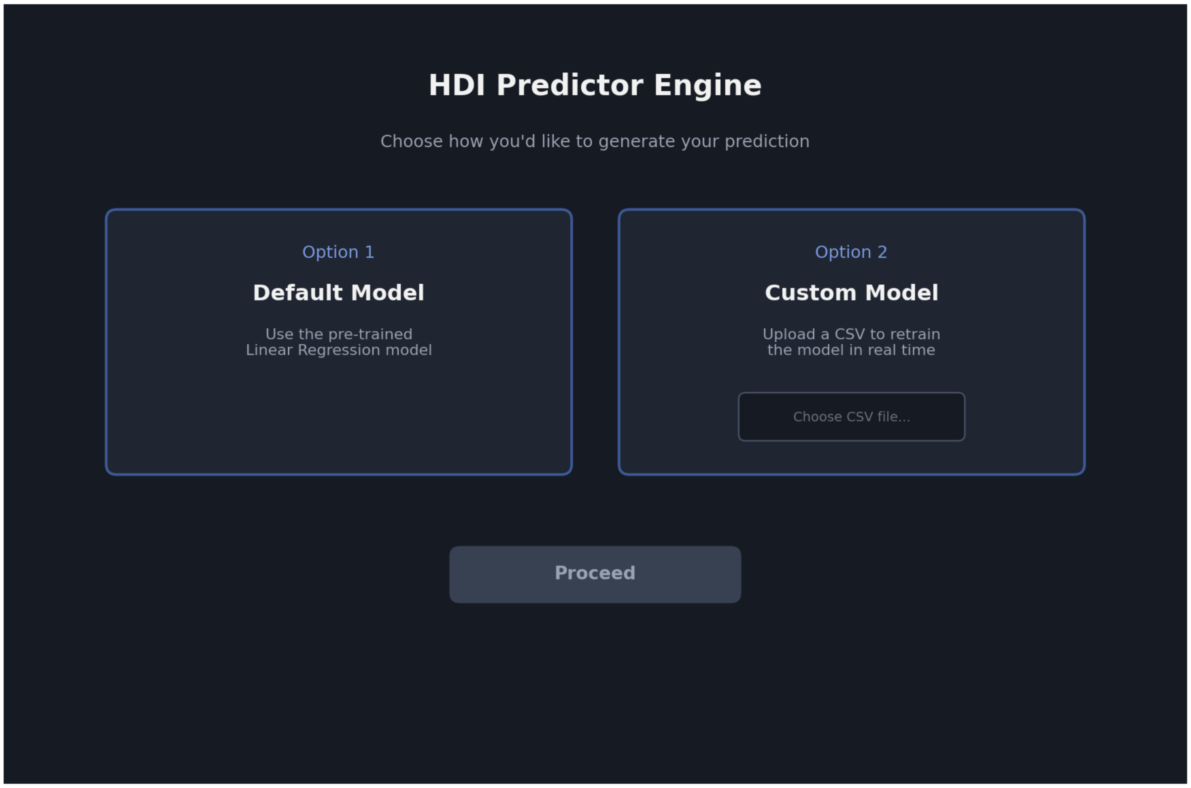
Activity 5.2: Public Vercel Deployment

- Configure routing in vercel.json and deploy using:

```
vercel --prod
```

Exploring the Web Pages

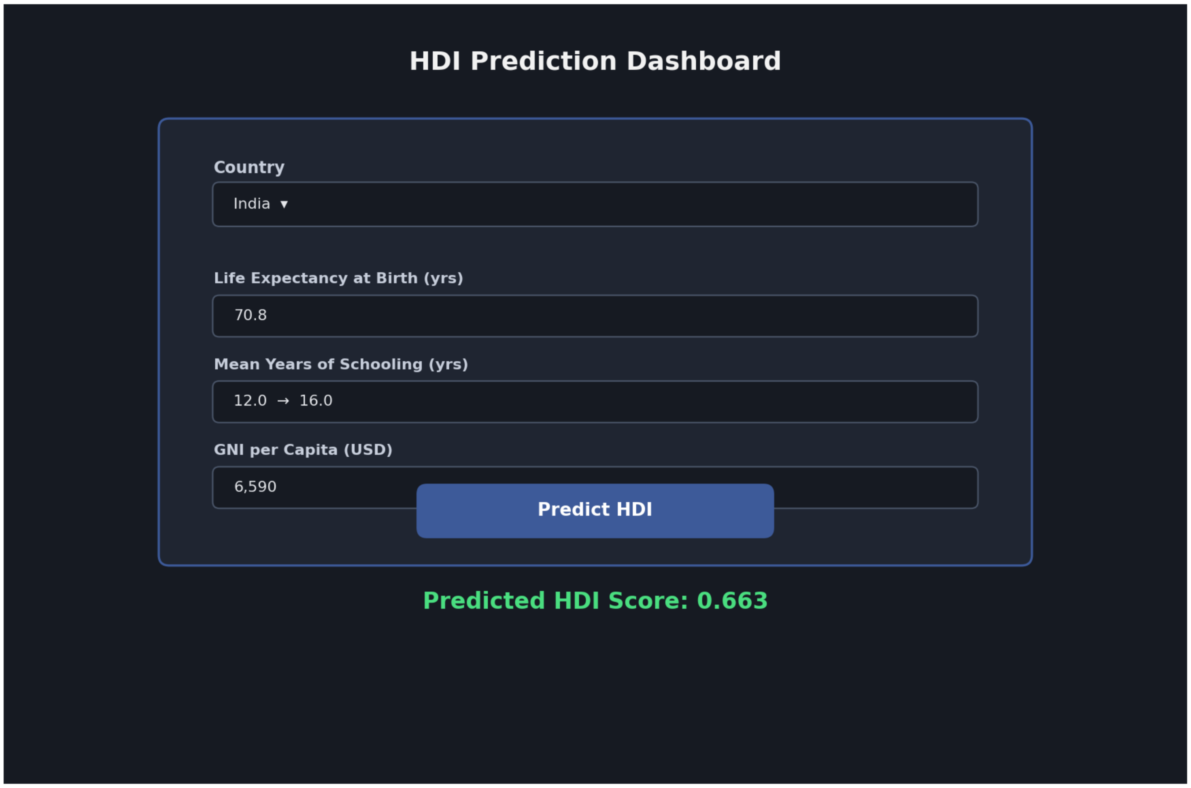
Home / Selection Page



Illustrative mockup of the home / selection page

Description: Features a clean, dark-themed interface with two options: Option 1: Default Model and Option 2: Custom Model (with file upload). The centralized button at the bottom remains disabled until a card is selected.

Prediction Page



Illustrative mockup of the prediction page

Description: Displays input fields (dropdown for Country, numbers for Life Expectancy, Schooling, GNI) and displays the predicted HDI score in green text at the bottom.

Conclusion & Future Extensions

The HDI Predictor is a serverless application that predicts national development statistics. Future extensions of this project include:

- **Live UN Database Syncing:** Integrating the World Bank Open Data API or UN API to automatically fetch the latest indicators.
- **Algorithm Comparisons:** Incorporating advanced estimators (Random Forest Regression) to compare prediction accuracy side-by-side.
- **Interactive Global Map:** Adding a geographical map dashboard using D3.js or Leaflet.js to visualize global predicted HDI scores dynamically.